# UNIVERSIDAD NACIONAL DE COLOMBIA

# Actividad 4: Genericidad, Excepciones y Archivos

Curso: Programación Orientada a Objetos (2026-2S) | Implementación GUI Python (Tkinter)



| | | | |
|---|---|---|---|
| **Estudiante:** | Kevin Santiago Montoya Chavarria | **Docente:** | Walter Hugo Arboleda Mazo |
| **Libro Guía:** | Leonardo Bermón Angarita (Capítulo 6) | **Repositorio GitHub:** | **github.com/Kevin007ch/POO-Kevin-Santiago-Montoya-Chavarria** |



## 1. RESUMEN EJECUTIVO DE LA ENTREGA

El presente informe documenta la adaptación de los ejercicios resueltos del Capítulo 6 del libro guía de Leonardo Bermón Angarita a la Programación Orientada a Objetos mediante **Python 3** y su librería nativa de interfaz gráfica **Tkinter**. Cada algoritmo de la lógica de negocio fue desacoplado en clases de dominio independientes y conectado a clases de vista gráfica ( `tk.Toplevel` / `tk.Tk` ). Los 5 módulos cubren el manejo de excepciones ( `try-except` ), la activación explícita con `raise` , la captura de excepciones múltiples ( `ValueError` , `ArithmeticError` ), la validación de formularios y la persistencia de lectura de archivos con captura de `IOError` .

## 2. MATRIZ DE EJERCICIOS RESUELTOS DEL LIBRO (CAPÍTULO 6 - PYTHON POO)

| Ejercicio Libro | Concepto Evaluado | Clase Lógica de Negocio | Clase Vista GUI (Tkinter) |
|---|---|---|---|
| **Ejercicio 6.4** | Manejo Básico de Excepciones | `CuentaBancaria` | `VentanaEjercicio6_4` |
| **Ejercicio 6.5** | Lanzamiento explícito ( `raise` ) | `ValidadorRango` | `VentanaEjercicio6_5` |
| **Ejercicio 6.6** | Bloques Catch/Except Múltiples | `CalculadoraEspecial` | `VentanaEjercicio6_6` |
| **Ejercicio 6.7** | Validación de Campos de Formulario | `FormularioCliente` | `VentanaEjercicio6_7` |
| **Ejercicio 6.8** | Lectura de Archivos ( `IOError` ) | `LectorArchivos` | `VentanaEjercicio6_8` |

## 3. DIAGRAMAS DE CASOS DE USO (UML)

**Casos de Uso del Sistema GUI en Python**

**Actor:** Usuario final interactuando mediante las ventanas de Tkinter.

- **CU01 - Gestionar Cuenta Bancaria (Ej 6.4):** Realizar consignaciones y retiros validando que no existan montos negativos o superiores al saldo disponible.
- **CU02 - Validar Rango de Entrada (Ej 6.5):** Evaluar valores numéricos según límites y lanzar excepciones capturadas por diálogos emergentes ( `messagebox` ).
- **CU03 - Operaciones Matemáticas (Ej 6.6):** Calcular logaritmos neperianos y raíces cuadradas capturando `ValueError` y `ArithmeticError` .

- **CU04 - Validar Datos de Formulario (Ej 6.7):** Verificar que no existan campos vacíos y que la edad esté en un rango válido.
- **CU05 - Leer Archivo de Texto (Ej 6.8):** Abrir archivos del sistema mediante `filedialog` y mostrar su contenido en un widget de texto.

## 4. DIAGRAMA DE CLASES (UML)

**Arquitectura de Clases en Python**

El proyecto mantiene una estructura orientada a objetos desacoplada:

- `AplicacionPrincipal(tk.Tk)` : Ventana principal de menú que inicia el bucle `mainloop()` y lanza las subclases `Toplevel` .
- `CuentaBancaria` ── (asociación) ──> `VentanaEjercicio6_4`
- `ValidadorRango` ── (método estático) ──> `VentanaEjercicio6_5`
- `CalculadoraEspecial` ── (método estático) ──> `VentanaEjercicio6_6`
- `FormularioCliente` ── (método estático) ──> `VentanaEjercicio6_7`
- `LectorArchivos` ── (método estático) ──> `VentanaEjercicio6_8`

## 5. CÓDIGO FUENTE E IMPLEMENTACIÓN DE LOS MÓDULOS

A continuación se resume la implementación clave en Python Orientado a Objetos registrada en el repositorio:

- **Módulo 6.4 ( `ejercicio6_4.py` ):** Define la clase `CuentaBancaria` con manejo de excepciones para depósitos y retiros.
- **Módulo 6.5 ( `ejercicio6_5.py` ):** Implementa `ValidadorRango.verificar()` mediante el lanzamiento explícito de `raise ValueError` .
- **Módulo 6.6 ( `ejercicio6_6.py` ):** Implementa `CalculadoraEspecial` con captura múltiple de `ValueError` y `ArithmeticError` .
- **Módulo 6.7 ( `ejercicio6_7.py` ):** Implementa `FormularioCliente.validar()` para la verificación integral de campos de texto y rangos de edad.
- **Módulo 6.8 ( `ejercicio6_8.py` ):** Implementa `LectorArchivos.leer_contenido()` gestionando la lectura e intercepción de `IOError` .

## 6. VERIFICACIÓN Y EVIDENCIA DE EJECUCIÓN

Los 5 programas han sido estructurados para ejecutarse desde el script principal `main.py` . Todas las excepciones son capturadas e informadas mediante diálogos `messagebox.showerror` o `messagebox.showwarning` , garantizando que la aplicación nunca se colapse por errores de usuario.